

myRecipes – Automatisches DB-Backup zu Dropbox

Strato vServer · /var/www/myrecipes/data/recipes.db → Dropbox:/Backup/rezepte · täglich 03:00 Uhr · max. 2 Saves

INHALT

1. Überblick & Voraussetzungen
2. cron auf dem Server installieren
3. rclone auf dem Server installieren
4. Dropbox-Token lokal generieren (Windows)
5. rclone auf dem Server konfigurieren
6. Backup-Skript erstellen
7. Cron-Job einrichten
8. Testen & Verifizieren
9. Wartung & Troubleshooting

1 · Überblick & Voraussetzungen

Ablauf

Täglich um 03:00 Uhr erstellt ein Shell-Skript via SQLite ein konsistentes DB-Snapshot, lädt es per

rclone

in einen Dropbox-Ordner hoch und löscht automatisch ältere Backups, sodass immer genau

2 Saves

erhalten bleiben.

Parameter	Wert
DB-Quelle (Server)	/var/www/myrecipes/data/recipes.db
Dropbox-Zielordner	dropbox:Backup/rezepte
Dateiname	recipes_YYYYMMDD_HHMMSS.db
Zeitplan	täglich 03:00 Uhr (Server-Zeit)
Aufbewahrung	2 neueste Backups, ältere werden gelöscht
Tool	rclone (offizielles CLI, aktiv gepflegt)

Voraussetzungen:

- SSH-Zugang zum Strato vServer (`root@82.165.153.177`)
- Dropbox-Konto (dein bestehendes)
- Windows: rclone Portable einmalig herunterladen für Token-Generierung
- `sqlite3` CLI auf dem Server (meist vorinstalliert, Fallback: `cp`)

2 · cron auf dem Server installieren

⚠ Wichtig: Strato vServer ist ein Minimal-System

Der Strato vServer kommt ohne vorinstallierten Cron-Daemon. Dieser muss zuerst manuell installiert werden, bevor der automatische Backup-Zeitplan funktioniert.

SERVER Per SSH einloggen und cron installieren:

```
ssh -i ~/.ssh/strato_key -o StrictHostKeyChecking=no root@82.165.153.177
```

```
# cron installieren
apt-get update && apt-get install -y cron

# cron-Dienst starten und für Autostart aktivieren
systemctl enable cron
systemctl start cron

# Status prüfen - muss "active (running)" zeigen
systemctl status cron --no-pager
```

Erwartete Ausgabe

Active: active (running) — erst dann läuft der automatische Zeitplan zuverlässig.

3 · rclone auf dem Server installieren

```
# Offizielles Installations-Skript (installiert in /usr/local/bin/rclone)
curl https://rclone.org/install.sh | sudo bash

# Prüfen
rclone --version
```

Tip: Manuelle Installation (falls curl-Skript geblockt)

```
wget https://downloads.rclone.org/rclone-current-linux-amd64.zip
unzip rclone-current-linux-amd64.zip
sudo cp rclone-*-linux-amd64/rclone /usr/local/bin/
sudo chmod 755 /usr/local/bin/rclone
```

4 · Dropbox-Token lokal generieren (Windows)

Da der Server kein GUI hat, wird der OAuth-Token auf deinem Windows-PC generiert und dann auf den Server übertragen.

4.1 · rclone Portable auf Windows herunterladen

LOKAL Öffne im Browser:

```
https://rclone.org/downloads/
```

→ Unter **Windows** die ZIP-Datei für **AMD64 / Intel/AMD - 64 Bit** herunterladen und entpacken (z.B. nach `C:\rclone\`).

4.2 · Token erzeugen

LOKAL

PowerShell öffnen und ausführen:

```
C:\rclone\rclone.exe authorize "dropbox"
```

Was passiert:

1. Der Browser öffnet sich automatisch mit der Dropbox-OAuth-Seite
2. Mit Dropbox-Konto einloggen und **"Zulassen"** klicken
3. Im PowerShell-Fenster erscheint der Token als JSON, z.B.:

```
{"access_token":"sl.ABC...", "token_type":"bearer", "expiry":"..."}
```
4. **Dieses JSON komplett kopieren** (Zwischenablage) – wird in Schritt 5 benötigt

⚠ Token sofort sichern

Das Token-Fenster schließt sich nach kurzer Zeit. Falls es weg ist, Schritt 4.2 einfach wiederholen.

5 · rclone auf dem Server konfigurieren

SERVER

Interaktive Konfiguration starten:

```
rclone config
```

Den Dialogen folgen – Eingaben sind **gelb** markiert:

```

No remotes found, make a new one?
n) New remote → n

name> → dropbox

Storage type (number or name):
→ dropbox # oder die Nummer für Dropbox eingeben

client_id> → [Enter] # leer lassen
client_secret> → [Enter] # leer lassen

Edit advanced config? → n

Use auto config? (headless machine → no) → n

For this to work, you will need rclone available on a machine that has a web browser available.
Execute the following on the machine with the web browser (same rclone version recommended):
  rclone authorize "dropbox" ...
Then paste the result below:

result> → [Token aus Schritt 4.2 einfügen]

Keep this "dropbox" remote? → y
q) Quit config → q

```

5.3 · Verbindung testen

```

# Dropbox-Root auflisten (sollte keine Ordner zeigen)
rclone ls dropbox:

# Zielordner anlegen (falls noch nicht vorhanden)
rclone mkdir dropbox:Backup/rezepte

# Test-Upload
echo "test" > /tmp/rclone_test.txt
rclone copy /tmp/rclone_test.txt dropbox:Backup/rezepte/
rclone ls dropbox:Backup/rezepte/
rclone delete dropbox:Backup/rezepte/rclone_test.txt
rm /tmp/rclone_test.txt

```

Erwartetes Ergebnis

`rclone ls dropbox:` zeigt keine Dropbox-Ordner. Nach dem Test-Upload zeigt `rclone ls dropbox:Backup/rezepte/` kurz die Datei, danach leer.

6 · Backup-Skript erstellen

SERVER Skript-Datei anlegen:

```
nano /usr/local/bin/backup-recipes-db.sh
```

Folgenden Inhalt einfügen:

```
#!/bin/bash
# -----
# myRecipes - Automatisches DB-Backup zu Dropbox
# Läuft täglich per Cron. Hält max. 2 Backups vor.
# -----

DB_PATH="/var/www/myrecipes/data/recipes.db"
DEST="dropbox:Backup/rezepte"
MAX_BACKUPS=2
STAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="recipes_${STAMP}.db"
TMP_FILE="/tmp/${BACKUP_FILE}"

echo "[$(date)] Starte Backup: ${BACKUP_FILE}"

# 1. Konsistentes DB-Snapshot (SQLite Online-Backup)
if command -v sqlite3 &>/dev/null; then
    sqlite3 "${DB_PATH}" ".backup ${TMP_FILE}"
else
    cp "${DB_PATH}" "${TMP_FILE}"
fi

if [ ! -f "${TMP_FILE}" ]; then
    echo "[$(date)] FEHLER: Backup-Datei konnte nicht erstellt werden"
    exit 1
fi

# 2. Zu Dropbox hochladen
rclone copy "${TMP_FILE}" "${DEST}"
if [ $? -ne 0 ]; then
    echo "[$(date)] FEHLER: Upload zu Dropbox fehlgeschlagen"
    rm -f "${TMP_FILE}"
    exit 1
fi

# 3. Temp-Datei aufräumen
rm -f "${TMP_FILE}"

# 4. Alte Backups löschen (nur neueste MAX_BACKUPS behalten)
COUNT=$(rclone lsf "${DEST}" --files-only | wc -l)
if [ "${COUNT}" -gt "${MAX_BACKUPS}" ]; then
    DELETE=$((COUNT - MAX_BACKUPS))
    echo "[$(date)] Lösche ${DELETE} alte Backup(s)..."
    rclone lsf "${DEST}" --files-only | sort | head -n "${DELETE}" | while read -r f; do
        echo "  → Lösche: ${f}"
        rclone delete "${DEST}/${f}"
    done
fi

echo "[$(date)] Backup erfolgreich: ${BACKUP_FILE}"
echo "[$(date)] Aktuelle Backups in Dropbox:"
rclone lsf "${DEST}" --files-only | sort
```

Skript ausführbar machen:

```
# Execute-Recht für den Owner (root) setzen
chmod u+x /usr/local/bin/backup-recipes-db.sh

# Prüfen
ls -la /usr/local/bin/backup-recipes-db.sh
# Erwartet: -rwxr--r-- ... backup-recipes-db.sh
```

Hinweis: Warum `sqlite3 .backup` statt `cp` ?

SQLite's `.backup`-Befehl nutzt die interne Hot-Backup-API – er erzeugt einen konsistenten Snapshot auch während das Node.js-Backend aktiv schreibt (WAL-Mode). Ein einfaches `cp` könnte bei gleichzeitigem Schreibzugriff eine inkonsistente Kopie erstellen. Das Skript fällt automatisch auf `cp` zurück, falls `sqlite3` nicht installiert ist.

7 · Cron-Job einrichten

SERVER Root-Crontab öffnen:

```
crontab -e
```

Zeile am Ende einfügen:

```
# myRecipes DB-Backup täglich 03:00 Uhr → Dropbox
0 3 * * * /usr/local/bin/backup-recipes-db.sh >> /var/log/recipes-backup.log 2>&1
```

Speichern und beenden (`Ctrl+X`, dann `Y`, dann `Enter` bei nano).

Crontab-Eintrag verifizieren:

```
crontab -l
```

Cron-Syntax erklärt:

`0 3 * * *` = Minute 0, Stunde 3, jeden Tag, jeden Monat, jeden Wochentag
→ täglich um
03:00 Uhr Server-Zeit

8 · Testen & Verifizieren

8.1 · Skript manuell testen

```
# Direkt ausführen und Ausgabe beobachten
/usr/local/bin/backup-recipes-db.sh

# Erwartete Ausgabe (Beispiel):
# [Tue May 19 03:00:01 UTC 2026] Starte Backup: recipes_20260519_030001.db
# [Tue May 19 03:00:04 UTC 2026] Backup erfolgreich: recipes_20260519_030001.db
# [Tue May 19 03:00:04 UTC 2026] Aktuelle Backups in Dropbox:
# recipes_20260519_030001.db
```

8.2 · Dropbox-Inhalt prüfen

```
rclone ls dropbox:Backup/rezepte/
```

8.3 · Skript zweimal ausführen (Rotation testen)

```
# Zweites Backup erzeugen (1 Sekunde warten für anderen Timestamp)
sleep 1 && /usr/local/bin/backup-recipes-db.sh

# Drittes Backup erzeugen → ältestes sollte gelöscht werden
sleep 1 && /usr/local/bin/backup-recipes-db.sh

# Jetzt genau 2 Dateien vorhanden:
rclone ls dropbox:Backup/rezepte/
```

8.4 · Log-Datei prüfen

```
# Nach dem ersten echten Cron-Lauf (nach 03:00 Uhr)
cat /var/log/recipes-backup.log
```

9 · Wartung & Troubleshooting

Problem	Lösung
<code>rclone: command not found</code>	Installation prüfen: <code>which rclone</code> Ggf. neu installieren: Abschnitt 3
Cron läuft nicht / kein Log	<code>systemctl status cron</code> — falls nicht aktiv: <code>apt-get install -y cron && systemctl enable --now cron</code>
Dropbox-Auth abgelaufen (Token invalid)	<code>rclone config reconnect dropbox:</code>
Upload schlägt fehl (Netz/Timeout)	Log prüfen: <code>tail -50 /var/log/recipes-backup.log</code> rclone nutzt automatische Wiederholversuche
<code>sqlite3</code> nicht vorhanden	Skript nutzt <code>cp</code> als Fallback – oder installieren: <code>apt-get install sqlite3</code>
Backup-Datei 0 Bytes	DB-Pfad prüfen: <code>ls -lh /var/www/myrecipes/data/recipes.db</code>
Permission denied beim Ausführen	<code>chmod u+x /usr/local/bin/backup-recipes-db.sh</code>

Log-Rotation einrichten (optional)

```
nano /etc/logrotate.d/recipes-backup
```

```
/var/log/recipes-backup.log {  
    weekly  
    rotate 4  
    compress  
    missingok  
    notifempty  
}
```

Backup manuell wiederherstellen

```
# Verfügbare Backups auflisten  
rclone ls dropbox:Backup/rezepte/  
  
# Gewünschtes Backup herunterladen  
rclone copy dropbox:Backup/rezepte/recipes_YYYYMMDD_HHMMSS.db /tmp/  
  
# App stoppen  
systemctl stop myrecipes  
  
# DB ersetzen (aktuelle zuerst sichern!)  
cp /var/www/myrecipes/data/recipes.db /var/www/myrecipes/data/recipes.db.bak  
cp /tmp/recipes_YYYYMMDD_HHMMSS.db /var/www/myrecipes/data/recipes.db  
  
# App starten  
systemctl start myrecipes
```

Zusammenfassung: Was auf dem Server installiert/konfiguriert wurde

- `cron` — Cron-Daemon (apt-get install cron)
- `/usr/local/bin/rclone` — rclone Binary
- `/root/.config/rclone/rclone.conf` — Dropbox-Konfiguration mit Token
- `/usr/local/bin/backup-recipes-db.sh` — Backup-Skript (chmod u+x)
- `crontab` (root) — Täglicher Aufruf um 03:00 Uhr
- `/var/log/recipes-backup.log` — Log-Datei (wird automatisch erstellt)
- `dropbox:Backup/rezepte/` — Dropbox-Zielordner